



University of Sousse
Higher School of Science and Technology of Hammam Sousse

Test Validation Plan (PVT)

PRO-PFE Project
Computer Science - LI2

Group Number: 17

Project Title: [Plateforme de recommandation des films et series](#)

Team Members:

[Ayoub Ramchani](#)
[Seif Handous](#)
[Akram Bel Hadj Yahia](#)

Academic Year 2025–2026

Contents

1 Règles obligatoires (Mandatory Rules):	3
2 Objectives du Testing	3
3 Scope of Testing	4
3.1 Included Features	4
3.2 Excluded Features	4
4 Stratégie de test	4
5 Outils de test	5
5.1 Tests unitaires	5
5.2 Tests de performance	7
5.3 Tests de robustesse	8
5.4 Tests de sécurité	9
6 Scénarios de test	9
7 Données de test	12
8 Critères de validation	13

Résumé exécutif

Ce document présente le **Plan de Validation des Tests (PVT)** du projet **Plateforme de recommandation de films et séries**.

Description du projet :

Le projet consiste à développer une plateforme de recommandation de films et séries permettant aux utilisateurs de rechercher des films, consulter leurs détails et recevoir des recommandations personnalisées basées sur leur historique et leurs préférences. Le système utilise une base de données pour la gestion des films, des favoris et des avis, ainsi qu'un moteur de scoring pour améliorer la pertinence des recommandations.

Ce document définit :

- La stratégie de test adoptée
- Les types de tests à effectuer (unitaires, robustesse, performance, sécurité)
- Les outils et l'environnement de test utilisés
- Les critères de validation des tests

1 Règles obligatoires (Mandatory Rules):

Ce projet de plateforme de recommandation de films et séries repose sur quatre catégories principales de tests, choisies afin de garantir la fiabilité, la sécurité et la performance du système dans un contexte réel d'utilisation :

- **Tests unitaires:** PHP (backend), couvrant les fonctions isolées de recherche, de scoring et de filtrage .
- **Tests de performance:** évaluation de la rapidité de traitement, du temps de réponse des requêtes et de la capacité de la plateforme à supporter une charge utilisateur élevée.
- **Tests de robustesse:** validation du comportement du système face à des données invalides, vides ou incohérentes.
- **Tests d'injection SQL (SQLMap):** analyse de sécurité visant à vérifier la résistance de la plateforme aux attaques par injection SQL sur les différents points d'entrée (recherche, authentification et API).

2 Objectives du Testing

The objectives of testing are:

- S'assurer la stabilité globale de l'application.
- S'assurer que le système répond correctement aux exigences fonctionnelles (recherche, recommandation et filtrage des films)
- Vérifier la fiabilité des fonctions backend à travers les tests unitaires
- Détecter et corriger les erreurs logiques avant le déploiement du système
- Garantir la robustesse de l'application face aux données invalides, vides ou inattendues
- Assurer la sécurité de la plateforme contre les attaques par injection SQL et les tentatives d'exploitation de la base de données
- Évaluer la performance du système sous charge et mesurer le temps de réponse des requêtes
- Vérifier la stabilité globale de la plateforme lors de l'utilisation simultanée de plusieurs utilisateurs
- Garantir la qualité et la cohérence des résultats de recommandation fournis aux utilisateurs

3 Scope of Testing

3.1 Included Features

Les tests réalisés dans le cadre de la plateforme de recommandation de films couvrent les fonctionnalités essentielles suivantes :

- Système de recherche de films (filtrage, mots-clés, pertinence des résultats)
- Moteur de recommandation basé sur le scoring et l'historique utilisateur
- Gestion des données en base (films, favoris, avis, utilisateurs)
- Fonctions backend développées en PHP (tests unitaires avec PHPUnit)
- Sécurité des entrées utilisateur contre les injections SQL (SQLMap)
- Gestion des cas limites et des erreurs (tests de robustesse)
- Performance des pages principales sous différentes charges utilisateur
- Intégration avec les API externes (ex : TMDB pour les films et posters)

3.2 Excluded Features

Certaines fonctionnalités ont été exclues du périmètre de test pour des raisons de priorité, de complexité ou de dépendance externe :

- Interface graphique avancée (UI/UX non testée en profondeur)
Les tests se concentrent sur la logique fonctionnelle plutôt que sur l'aspect visuel.
- Tests utilisateurs réels (UX testing)
Non réalisés faute de panel utilisateur et hors périmètre technique du projet.
- Tests de compatibilité multi-navigateurs avancés
L'application est validée dans un environnement standard de développement.
- Tests de sécurité avancés (pentesting complet)
Seule la vulnérabilité aux injections SQL a été couverte dans le cadre académique.
- Tests approfondis des API externes (TMDB)
Les API externes sont considérées comme des services fiables et indépendants du projet.

4 Stratégie de test

Les tests seront réalisés à différents niveaux afin de garantir la fiabilité, la sécurité et la performance de la plateforme de recommandation de films :

- Tests unitaires (validation au niveau du code)
- Tests d'intégration (interaction entre les composants tels que le backend, la base de données et les API externes)

- Tests système (scénarios complets de bout en bout)

Explication importante :

- Les tests unitaires assurent la correction des fonctions individuelles du backend telles que le scoring, le filtrage et la logique de recommandation implémentée en PHP (PHPUnit)
- Les tests d'intégration valident l'interaction entre les différents composants du système, notamment les requêtes API, les opérations sur la base de données et les services externes comme TMDB
- Les tests système vérifient le bon déroulement complet du parcours utilisateur, depuis la recherche de films jusqu'à la consultation et l'utilisation des recommandations

5 Outils de test

Les outils de test utilisés dans ce projet de plateforme de recommandation de films ont été sélectionnés en fonction des besoins de validation fonctionnelle, de sécurité et de performance :

- PHPUnit – Tests unitaires du backend PHP (validation des fonctions de recherche, scoring et recommandation)
- Composer – Gestion des dépendances PHP et environnement de test
- Python (scripts de test) – Tests de performance et simulation de charge utilisateur
- XAMPP – Environnement local de développement et d'exécution du serveur web et de la base de données
- MySQL – Base de données utilisée pour les tests d'intégrité et de cohérence des données
- Navigateur web (Chrome / Firefox) – Tests fonctionnels de l'interface utilisateur et des requêtes AJAX

5.1 Tests unitaires

Objectif : Vérifier le bon fonctionnement des différentes fonctions du backend de manière isolée.

Explication : Les tests unitaires permettent de valider la logique interne du système de recommandation de films, notamment les opérations sur la base de données, les algorithmes de scoring, les règles de filtrage ainsi que les composants d'affichage. Ils sont réalisés avec PHPUnit et garantissent que chaque module fonctionne correctement indépendamment du reste du système.

Description des fonctionnalités testées :

- Base de données (Database)

- Vérification que les identifiants des films sont positifs et auto-incrémentés
- Ajout de films aux favoris sans duplication
- Calcul correct de la moyenne des avis utilisateurs
- Gestion de l'insertion de films avec champs optionnels manquants
- Test de suppression en cascade des avis associés aux films (cas d'échec détecté)
- **Moteur de scoring (FilmSearchScorer)**
 - Calcul du score basé sur les mots-clés, genres, acteurs, réalisateurs et langues
 - Vérification des correspondances partielles et exactes
 - Gestion des cas invalides (chaînes vides, valeurs nulles, formats incorrects)
 - Détection de comportements incohérents dans certains cas d'acteurs ou de formats
- **Interface utilisateur**
 - Présence des éléments essentiels (barre de recherche, dropdown, wrapper)
 - Fonctionnement de la recherche dynamique (live search)
 - Navigation vers la page de description des films
 - Disponibilité du bouton de lecture et de consultation
- **Moteur de recommandation**
 - Absence de recommandations sans historique utilisateur
 - Génération de recommandations basées sur l'historique
 - Exclusion des films déjà visionnés
 - Classement des films selon la pertinence
 - Prise en compte des profils utilisateur (genres et acteurs)
- **Cache TMDB**
 - Récupération correcte des informations (poster, ID TMDB)
 - Gestion des films inconnus (valeurs nulles)
 - Création et réutilisation du cache
 - Gestion des fichiers JSON corrompus
 - Robustesse face aux erreurs d'écriture du cache

Conclusion : Les tests unitaires ont permis de valider la majorité des composants critiques du système. Cependant, certains cas d'échec ont été détectés, notamment dans la gestion des formats d'acteurs, la suppression en cascade et certaines incohérences dans le scoring, nécessitant des améliorations futures.

5.2 Tests de performance

Objectif : Mesurer le comportement du système sous différentes charges utilisateur.

Explication : Les tests de performance permettent d'évaluer la capacité de la plateforme de recommandation de films à maintenir de bonnes performances lors d'un grand nombre de requêtes simultanées. Ces tests analysent notamment :

- Le temps de réponse des pages principales
- La stabilité du système sous forte charge
- La capacité maximale du serveur
- Le nombre de requêtes traitées par seconde
- Le comportement du backend et des requêtes AJAX lors d'utilisations simultanées

Les tests ont été réalisés à l'aide d'un script Python asynchrone utilisant la bibliothèque `aiohttp`. Ce script simule plusieurs utilisateurs virtuels accédant simultanément à la plateforme.

Pour chaque niveau de concurrence, plusieurs requêtes HTTP sont envoyées vers les principales pages de l'application afin de mesurer la latence moyenne, le débit et les éventuelles erreurs retournées par le serveur.

Charges testées :

- 10 utilisateurs simultanés
- 100 utilisateurs simultanés
- 1000 utilisateurs simultanés

Fonctionnalités évaluées :

- Chargement de la page d'accueil
- Réactivité du moteur de recherche
- Temps de réponse des requêtes backend PHP
- Stabilité des connexions à la base de données
- Gestion des requêtes multiples simultanées

Approche utilisée :

- Simulation de plusieurs utilisateurs virtuels
- Mesure du temps total d'exécution des requêtes
- Calcul de la latence moyenne par requête
- Analyse du débit (requêtes/seconde)
- Détection des erreurs serveur sous forte charge

Remarque : Ces tests permettent d'identifier les limites de performance du système et les composants nécessitant des optimisations, notamment au niveau des requêtes SQL, du backend PHP et de la gestion des accès concurrents.

5.3 Tests de robustesse

Objectif : Vérifier la stabilité du système face à des données invalides, imprévues ou extrêmes.

Explication : Les tests de robustesse permettent d'évaluer la capacité de la plateforme de recommandation de films à gérer correctement les erreurs, les valeurs nulles, les formats incorrects ainsi que les cas limites sans provoquer de panne ou de comportement critique.

Ces tests valident la résistance du système dans différentes situations inhabituelles pouvant être générées par les utilisateurs, les requêtes AJAX ou les données provenant des API externes.

Fonctionnalités testées :

- Gestion des profils utilisateurs vides ou inexistants
- Vérification des identifiants utilisateur nuls
- Gestion des films vides dans le moteur de recommandation
- Validation des recherches sans critères ou contenant uniquement des espaces
- Gestion des mots-clés inexistants dans la recherche
- Vérification des descriptions incomplètes (vote null, overview null, poster absent)
- Gestion des réponses invalides provenant de l'API TMDB
- Validation des requêtes AJAX trop courtes ou vides
- Nettoyage des entrées utilisateur avec trim
- Contrôle des dépassements de score (overflow)
- Validation des formats d'acteurs incorrects
- Gestion des duplications de favoris dans la base de données

Approche utilisée :

- Exécution de scénarios contenant des données invalides ou incohérentes
- Simulation d'erreurs provenant du backend et des API externes
- Vérification du comportement du système face aux valeurs nulles
- Contrôle de la stabilité des requêtes AJAX et des opérations SQL
- Détection des exceptions et des violations de contraintes dans la base de données

Analyse générale :

Les tests de robustesse montrent que le système gère correctement la majorité des cas limites et des erreurs utilisateur. Cependant, certaines anomalies ont été détectées, notamment dans la gestion des dépassements de score, des formats d'acteurs invalides et des duplications de favoris en base de données. Ces résultats permettent d'identifier les parties du système nécessitant des améliorations supplémentaires afin de renforcer la stabilité globale de la plateforme.

5.4 Tests de sécurité

Objectif : Vérifier la sécurité de la plateforme contre les attaques ciblant la base de données et les entrées utilisateur.

Explication : Les tests de sécurité permettent d'identifier les vulnérabilités potentielles dans l'application web, notamment les failles d'injection SQL pouvant compromettre les données utilisateurs ou la base de données du système.

Dans ce projet, les tests de sécurité ont été réalisés à l'aide de l'outil **SQLMap**, un framework open source spécialisé dans la détection automatique des vulnérabilités SQL Injection.

Ces tests permettent d'évaluer la capacité de la plateforme à résister aux tentatives d'exploitation malveillantes via les formulaires, les paramètres URL et les requêtes HTTP.

Fonctionnalités testées :

- Validation des entrées utilisateur dans les formulaires de recherche
- Vérification des paramètres transmis via les requêtes GET et POST
- Protection des requêtes SQL exécutées par le backend PHP
- Résistance de la base de données face aux tentatives d'injection SQL
- Vérification des accès aux données sensibles
- Contrôle des interactions entre le backend et MySQL

Approche utilisée :

- Utilisation de SQLMap pour analyser automatiquement les paramètres vulnérables
- Simulation de requêtes malveillantes sur les points d'entrée du système
- Vérification des réponses du serveur face aux injections SQL
- Contrôle des messages d'erreur retournés par l'application
- Analyse des protections mises en place dans les requêtes SQL

Importance des tests de sécurité :

Ces tests sont essentiels pour garantir la confidentialité et l'intégrité des données de la plateforme de recommandation de films. Ils permettent de réduire les risques d'accès non autorisé à la base de données et de renforcer la sécurité globale du système face aux attaques web les plus courantes.

6 Scénarios de test

Scénario TS-01 – Recherche de film

Préconditions : La base de données contient plusieurs films enregistrés.

Étapes :

1. Ouvrir la page d'accueil

2. Saisir un mot-clé dans la barre de recherche

3. Lancer la recherche

Résultat attendu : Le système affiche une liste de films correspondant au mot-clé recherché.

Priorité :

- Critique

Scénario TS-02 – Recherche avec champ vide

Préconditions : L'utilisateur se trouve sur la page de recherche.

Étapes :

1. Laisser le champ de recherche vide

2. Cliquer sur le bouton de recherche

Résultat attendu : Le système ne plante pas et affiche un message approprié ou une liste vide.

Priorité :

- Majeure

Scénario TS-03 – Génération de recommandations

Préconditions : L'utilisateur possède un historique de films consultés.

Étapes :

1. Se connecter à la plateforme

2. Accéder à la page des recommandations

Résultat attendu : Le système génère des recommandations pertinentes basées sur l'historique de l'utilisateur.

Priorité :

- Critique

Scénario TS-04 – Gestion des favoris

Préconditions : L'utilisateur est connecté à la plateforme.

Étapes :

1. Ajouter un film aux favoris

2. Essayer d'ajouter le même film une deuxième fois

Résultat attendu : Le système empêche la duplication des favoris.

Priorité :

- Majeure

Scénario TS-05 – Gestion des données invalides

Préconditions : Le système est en cours d'exécution.

Étapes :

1. Envoyer des valeurs nulles ou incorrectes dans une requête
2. Observer le comportement du système

Résultat attendu : Le système reste stable et gère correctement les erreurs sans interruption.

Priorité :

- Critique

Scénario TS-06 – Recherche AJAX (saisie courte)

Préconditions : La page d'accueil est ouverte.

Étapes :

1. Saisir moins de deux caractères dans la barre de recherche
2. Observer le comportement de la recherche dynamique

Résultat attendu : Le système ignore ou bloque les requêtes trop courtes.

Priorité :

- Majeure

Scénario TS-07 – Test de performance sous charge

Préconditions : Le serveur local XAMPP est démarré.

Étapes :

1. Exécuter le script Python de test de performance
2. Simuler plusieurs utilisateurs simultanés
3. Mesurer le temps de réponse du système

Résultat attendu : Le système reste stable et répond dans un temps acceptable sous charge.

Priorité :

- Critique

Scénario TS-08 – Test de sécurité (injection SQL)

Préconditions : Le serveur et la base de données sont actifs.

Étapes :

1. Tester les points d'entrée utilisateur avec des requêtes malveillantes
2. Analyser les réponses du serveur

Résultat attendu : Le système ne doit pas être vulnérable aux injections SQL.

Priorité :

- Critique

7 Données de test

Explication : Les données de test représentent les entrées utilisées pour vérifier le bon fonctionnement de la plateforme de recommandation de films. Elles permettent de simuler différents comportements utilisateur ainsi que des cas valides, invalides et limites.

Exemples de données utilisées :

- **Utilisateurs de test :**

- Utilisateur standard avec historique de visionnage
- Utilisateur sans historique (nouveau profil)
- Utilisateur avec identifiant nul ou invalide

- **Données de recherche :**

- Mots-clés valides : *action, aventure, drame*
- Chaînes vides ou espaces uniquement
- Mots-clés inexistant dans la base de données

- **Données de films :**

- Films avec toutes les informations complètes (titre, genre, note)
- Films avec champs manquants (poster, overview, vote)
- Films avec identifiants invalides ou négatifs

- **Données de favoris :**

- Ajout d'un film en favori
- Tentative de duplication d'un favori existant
- Suppression ou gestion d'un favori inexistant

- **Données de robustesse :**

- Valeurs nulles et champs vides
- Formats invalides (acteurs ou scoring incorrect)
- Cas extrêmes dans les calculs de score

- **Données de performance :**

- Simulation de 10, 100 et 1000 utilisateurs simultanés
- Requêtes répétées vers la page d'accueil et la recherche

- **Données de sécurité :**

- Entrées potentiellement malveillantes dans les champs de recherche
- Tentatives d'injection SQL via les paramètres URL ou formulaires

8 Critères de validation

Un test est considéré comme réussi si :

- Le résultat obtenu correspond au résultat attendu
- Le système fonctionne correctement sans erreurs inattendues
- Le comportement de l'application reste stable dans tous les cas testés

Un test est considéré comme échoué si :

- Le résultat obtenu est incorrect ou incohérent par rapport aux spécifications
- Le système génère des erreurs ou des exceptions non prévues
- L'application se bloque ou présente un comportement instable
- Les règles métier ou de sécurité ne sont pas respectées